Progress Report

on

Vision Based Smart Warehousing and Inventory Management

A Project Based Learning Report Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology in Robotics and Automation

By

Sonika Das (23070127058)

Naman Agarwal (23070127037)

Ashmit Sanjay Katale (23070127012)

॥ वसुधैव कुटुम्बकम् ॥

Project Guide

Dr. Sameer Sayyad

Assistant Professor,

Symbiosis Institute of Technology, Pune

Department of Robotics and Automation

SYMBIOSIS INSTITUTE OF TECHNOLOGY,

A Constituent of Symbiosis International (Deemed University), Pune-412115

2025-26

Endorsement of Project Work

This is to certify that the Project Proposal titled **"Vision Based Smart Warehousing and Inventory Management"** has been carried out by the student group as part of Project Based Learning (PBL). The proposed work and documentation have been prepared as per the prescribed format, guidelines, and evaluation criteria of the Department and Institute. The project is approved for the **First Review / Project Proposal Presentation**.

Place:

Date: 24-01-2026

Name and Signature of Students	**Name and Signature of Guide**
Sonika Das (23070127058)	**Dr Sameer Sayyad** Assistant Professor, Department of Robotics and Automation, Symbiosis Institute of Technology, Pune
Naman Agarwal (23070127037)	
Ashmit Sanjay Katale (23070127012)	

Annexure 2

Undertaking for Literature review and Project proposal Submission

Department	**Robotics and Automation**
Batch	**2023-27**
Name of the Students (PRN)	1. Sonika Das (23070127058) 2. Naman Agrawal (23070127037) 3. Ashmit Sanjay Katale (23070127012)
Name of the guide(s):	Dr. Sameer Sayyad, Assistant Professor, Robotics and Automation Department
Project Title	Vision Based Smart Warehousing and Inventory Management
Similarity Index (should be less than 10%) AI Similarity (must be 0% or *%)	1. Similarity Index *(must be* ≤10%*)*: 2. AI Report *(must be* 0% *or* * %*)*:

Date:24-01-2026

Signature of Student(s):

1. Sonika Das
2. Naman Agrawal
3. Ashmit Sanjay Katale

Signature of Project Guide

Dr. Sameer Sayyad

1. Introduction

In retail environments today, the shelves stacked with various products and their individual flow of goods and information is hard to keep track of manually and hence automation of such monitoring at such a scale is a need of the hour. Up till now barcode scanner and RFID systems[1] have done all they can to make this process efficient but its limitations like requiring a specific line of sight for each product, its inability to track products until scanned at each terminal/junction, being prone to miss scans thus stock that may be mismatched on the shelves are all the reasons that barcode scanner systems are long outdated in this environment where tracking of inventory must be absolute and faultless. This can be accomplished with computer vision models that have the appropriate context of size and content of such products so that they can be tracked.[2, 3]

As the retail space grows so does the need for edge computing, signs of this are the emergence of systems like self-checkout systems that use AI-enabled cameras and models that keep track of not only individual carts but also the total inventory while crunching all this data in real-time.[2, 4]

In such a space, you would admit there is a dire need of low-cost embedded solutions to these problems for scalable retail deployment. And that's just what our project offers- a scalable robotic system that can keep track of not just individual customers' carts but also the entire inventory.

1.1 Background/Rationale

Retail spaces today experience constant product movement amongst several checkpoints- cart, shelf, checkout zones etc. due to the continuous browsing of products from shelves and their consecutive misplacement or replacement to the shelf as customers "window-shop" products.

Such applications require high accuracy as any blame for missing products can not be placed on the customer without evidence to avoid arbitrary litigation. Here barcode and RFID product tracking require a lot of infrastructure and assume orderly customer behaviour which is far from reality and not grounded in its assumptions of human behaviour.

On the other hand, stationery, vision based shelf monitoring systems enable automated real-time stock tracking, demand analysis and restocking alerts that are all initiated real-time.[5,6,7] They do not require movement or at least require very minimal movement to accommodate their entire visual space. This space constraint can also be tackled simply by acquiring the same space of a shelf or hallway of a retail space from multiple cameras placed to track products from all angles to ensure complete accuracy.

The human element of their interaction with shelves and their products could also be used to gain some very unique and small but important insights into a customer's buying patterns, if this data is acquired using edge computing to streamline buying pattern indicators, the purchase efficiency of each shelf and store could be optimized for maximum purchase.[2,4]

1.1.1 Motivation of the Study

The motivation for this study arises from the frequent retail changes and changes in inventory appearance over time. In such cases fixed automation systems may struggle to adapt swiftly and may end up causing breakdown or halting of these systems temporarily affecting profitability of the store. Even now self-checkout systems are prone to being fooled by customers who manage to hide items or are unable to scan the item properly due to occlusions or change in appearance of a product launched with new packaging that has not been updated in the system.

- Inventory checks are mostly manual now, due to which audits of these inventory can be time-consuming and tedious.
- Shelf layouts also change frequently to accommodate space for newer products while the older ones sit on the shelf taking space. This along with difficulty in achieving real-time accuracy with systems that are rigid to change and adapt independently are some of the operational motivations behind this project
- Existing solutions for such cases/ scenarios assume orderly and stable conditions meaning the models are trained once and fine-tuned occasionally to keep accuracy from declining but that still happens when conditions change
- **Role of Continual Learning in Inventory and Spatial Systems:**

  Inventory accuracy depends on repeated observation and as previously established static models' performance degrades over time, to combat this we integrate **Continuous Learning** which corresponds to adaptive perception that improves trust in collected data. Continual learning updates models gradually. Learning occurs without full retraining meaning safeguards can be installed from catastrophic forgetting while also supporting consistent inventory and mapping results.

1.1.2 Major Technologies

The major technologies involved in our project include-

1. Computer Vision for object detection, camera-based scene understanding
2. Continual learning for gradual model updates i.e Learning without forgetting old data and adaptability to new circumstances.
3. Fixed vision systems
4. Occupancy based shelf-understanding
5. Fully local/ on-device computation meaning no need for cloud re-training or cloud infra-structure.

1.2 Key Advantages

1. Less manual recalibration.
2. Better handling of layout changes.
3. Better handling of lighting variation using a LED of our own to consistently light subjects appropriately
4. Learning during operation.
5. No system shutdown for updates.
6. Stable perception over time.
7. Works on low-cost hardware

1.3 Recent Trends

- Static models fail in changing environments and need learning during deployment. Following this we noticed an Increase in continual learning research and a shift from cloud AI to edge AI application research.
- Vision-based automation is becoming common but perception quality and consistency drops over tim, this highlights the need for scalable automation solutions that focus on cost-effective systems for SMEs.[1,3,8]

1.4 Significance of Study

Our solution links research with real, ground-level deployment. Focus on practical retail space use and showcases adaptive robotic perception. It aims to reduce maintenance effort and constant retraining requirements thus improving long-term reliability.

All this while supporting continuous inventory monitoring.

1.5 Organization of Report

This structure of the report begins wiht identifying challenges in traditional retail invenmtory management and progresses consequently towards the development of the proposed vision based smart inventory management solution.

In the introductory phase, the report outlines the limitations and challenges of current solutions in the retail space, the growing demand for automation in a dynamic environment along with the significance of using computer vision coupled with continual learning for the purposes of smart inventory management.

Following that, an exhaustive and comprehensive review of the available research material examines prior research in topics including but not limited to object detection, edge-based AI systems, retail automation and continual learning frameworks. This section illuminates the technological advancements in the field while identifying key gaps in these fields.

The methodology section showcases the complete proposed system architecture, including hardware integration, software stack selection, the interactive layer (HMI) and the model training strategy i.e using EWC (ELastic Weight Consolidation) for continual learning. This section outlines in detail the perception learning feedback loop along with the phase-by-phase implementation plan. Subsequently the report discusses the work that has been accomplished so far- hardware assembly, the YOLOv8 object detection model, GUI development for the HMI and system integration. This section also encompasses the performance metrics namely- mAP, precision, recall, inference latency, and counting accuracy to analyze system effectiveness.

Finally the report concludes the key contributions of our proposed projects, emphasizes adaptability to dynamic environments and cost-effective deployment of retail automation. We also included some recommendations of our own for future improvement that we couldn't implement ourselves due to various constraints like budget and time.

2. Review of Relevant Literature

2.1 Previous Research/Work

Our general observations of the combination of retail space and vision based models confirmed for us that static models degrade in performance over time thus making them un-ideal for long term deployment due to their inability to comprehend environmental changes that causes data-distribution to shift.[9,10]

Foundational works in continual learning establish that traditional retraining-based approaches are inefficient and disruptive for systems requiring continuous operation. Research regarding robotic perception, mobile applications, retail data streams, and vision-based automation shows that incremental, on-device learning methods provide a more sustainable alternative. Review studies further indicate that continual learning has been explored across task-incremental, domain-incremental, and online learning settings, with increasing attention to real-world deployment constraints such as limited compute, memory, and uninterrupted operation, all the areas where memory retention is a key challenge in face of continuous retraining.

In the context of robotics and retail automation space, prior work on, object detection, and edge-based vision systems demonstrates the feasibility of autonomous perception and monitoring systems, but also reveals that although continuous learning is well studied in theory, its practical deployment has been limited.[2, 3, 11] Surveys of retail automation shows us that advanced vision algorithms have difficulty integrating into mainstream hardware, specifically embedded hardware.[1, 8] The studies we gathered will convincingly establish a solid grounding for adaptive, learning enabled robotic systems and highlight some major shortcomings within models that are currently being deployed.

2.2 Key Findings

The following are the conclusions we gathered from our extensive literary review:

- This is a problem common to static vision and learning systems that operate in non-stationary environments.
- The continuous learning approaches significantly reduce the loss of accuracy over time by incrementally adjusting the models in order to retain knowledge acquired.[9. 12]
- These strategies have strengths in combating catastrophic forgetting, but have computational and memory challenges that can only be further studied by practical deployment of such systems
- Domain-incremental and data-stream learning setups better reflect real-world industrial and robotic conditions compared to isolated task-based evaluations.
- On-device and edge-based learning is increasingly recognized as essential for privacy, latency, and cost-sensitive applications.[1, 3, 7]

Together these findings support the hypothesis that continuous learning is not an optimization, but a need for long term autonomous functioning within environments that are populated by human behaviour.

2.3 Research Gaps

Yet there remains a lack of clarity in the literature concerning several issues that are not addressed yet.

- Limited integration of continual learning with robotic platforms (mobile or static) performing real-world inventory or monitoring tasks.[13, 14]
- Insufficient focus on low-cost, embedded hardware constraints, particularly for small and medium-scale deployments i.e. SMEs.
- Lack of generic standards for perception accuracy and adaptation stability
- Few studies demonstrate end-to-end systems that combine continual learning and real-world deployment without reliance on offline retraining or cloud infrastructure.[5,6,15]

These gaps are essential for the proposed project, which aims to combine theory of continuing learning and practice using robotics in retail and inventory management.

3. Problem Statement and Objectives

3.1 Research Questions

This project is guided by the following research questions:

- Can Vision-Based Intelligent Systems remove the need for manual labour or sensor-heavy implementations; improving efficiency and cost-effectiveness?
- Can continual learning–based vision systems serve as a viable alternative to traditional sensor-heavy approaches for mapping and environment understanding?
- To what extent can task-specific, lightweight computer vision models enable long-term adaptability while remaining feasible on low-cost embedded platforms?

- Can a proof-of-concept robotic system effectively demonstrate continual learning principles without scaling computation beyond practical limits?

These questions focus on feasibility and adaptability rather than absolute performance.

3.2 Problem Statement

Traditional solutions are prone to fooling, errors caused by occlusions and/or introduction of products that the system may not have seen/ been trained on before.[11] They are also RFID or barcode based which require ordered scanning at several checkpoints and require expensive cloud based infra-structure.

Also, static perception models degrade over time and need constraint retraining to avoid accuracy drop which still happens because the inputs i.e their environment (lighting, product appearance, shelf layout) keep changing and they are un-equipped to deal with a dynamic environment.

These systems also require expensive hardware, cloud based learning and rigid models, all of which we feel these systems can do without. Instead they must be adapt-able and perform with low-cost embedded system components all while keeping their computation local i.e. edge computing for real-time application

3.3 Objectives

Our project, Vision-Based Smart Inventory Management System implements the following objectives:

- To detect multiple objects on a retail shelf using an appropriate computer vision–based algorithm.
- To accurately estimate the count of individual product categories present on the shelf from visual data.
- To design and develop a graphical user interface (GUI) that displays the real-time inventory status of items on the shelf.
- To design and integrate a dedicated embedded hardware platform capable of real-time image capture, processing, and continuous shelf monitoring in a retail environment.

3.4 Scope and Limitations

3.4.1 Scope

- Perception is limited to vision-based sensing, using task-specific datasets and models.
- Continual learning is applied at the perception level, rather than full end-to-end control.
- The system is intended as a research and demonstration platform, not a commercial-grade deployment.

3.4.2 Limitations

- This system does not employ high-precision sensors such as LiDAR, limiting mapping accuracy, instead focusing on adaptability and feasibility over large scale deployment using affordable embedded system components
- Limited computational resources constrain the scale of the models and the resolution at which I process images, as well as the number of learning updates.

4. Methodology

4.1 Overview of Methodology

Vision-Based Smart Warehousing and Inventory Management project that we have worked on is a closed loop intelligent system based on a continual learning paradigm that monitors shelves in retail environments and learns on the fly when prompted to do so. In this project, perception and learning are not separate modules but closely tied together in a closed loop. Two main stages have been developed:

- **Perception Phase:** The system perceives these changes and states of a shelf's products using a camera that is strategically placed in as much of the environment as it can. This achieves our second objective to detect multiple objects using computer vision and the third objective, that is, accurately estimate product counts.[2, 5]
- **Learning & Interaction Phase:** The system learns from new data over time and presents insights via an interface. Through this, we achieve the third objective to design a GUI for real-time inventory status on an HMI.

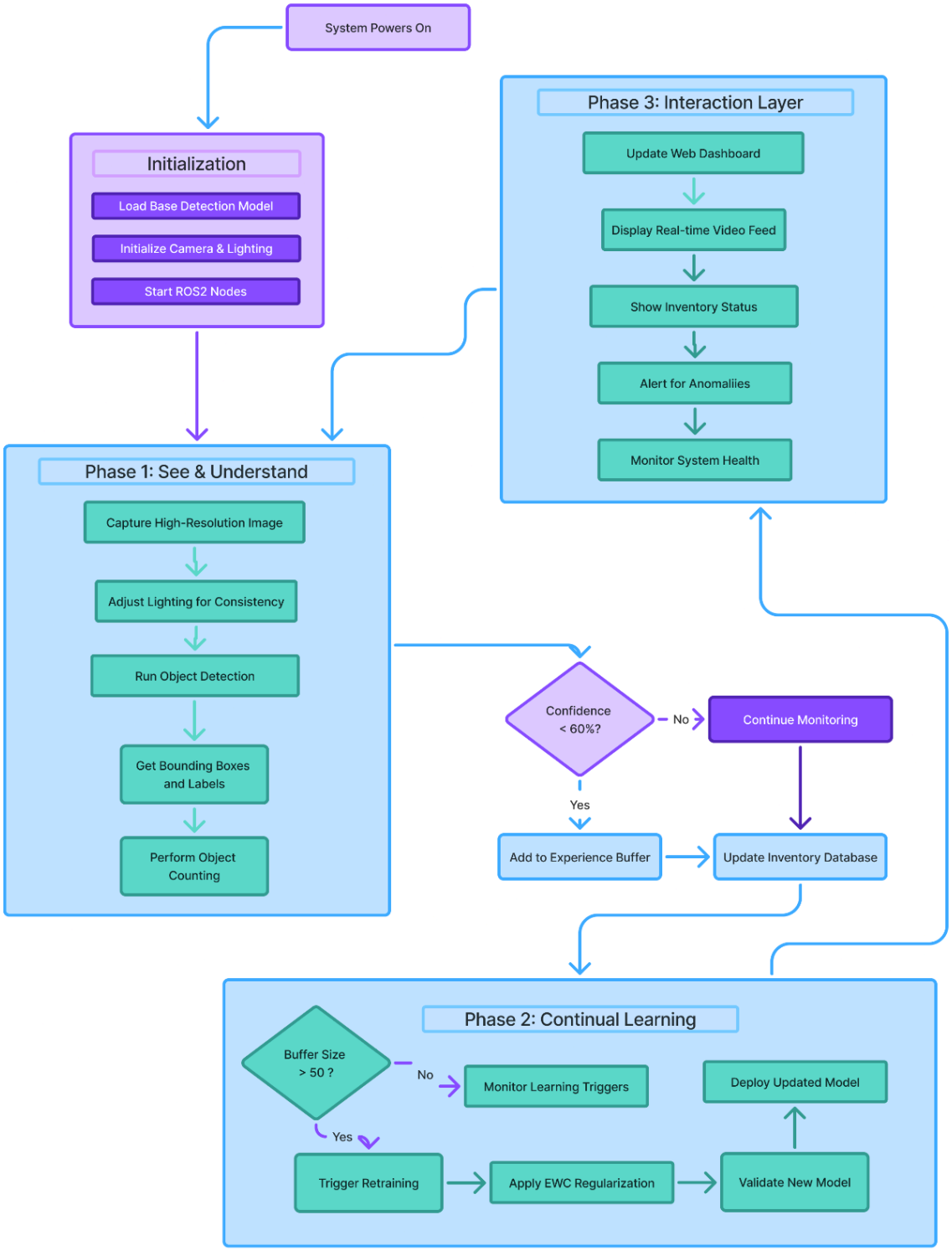


Figure 4.1: End-to-End Working of the System

4.2 Design

To design the system architecture of this project, we drew inspiration from the biological nervous system and decided on the following three-layers:

The Hardware Layer: comprises the static sensing unit. It is further divided into two parts: the Sensing System: a high-resolution camera module and an integrated LED light right to ensure consistent capturing of data critical for detection; and the Computation Unit: Raspberry Pi 5 to host all the software components.

The Computational Layer is the adaptive brain that carries complex algorithms. It is also sub-divided as follows:

- Perception System: Responsible for tasks such as object detection, depth estimation and counting algorithm.[3,5,16]
- Learning System: the Core where our continual learning model lies. Responsible for adaptive learning of the system that allows the model to adapt to new products or packaging without catastrophic forgetting
- GUI Backend: Responsible for managing the data flow to the UI dashboard and showcases detection results, alerts and inventory count.

The Interaction Layer is the interface for human operators. A full feedback loop between the machine intelligence and its human operators is thus achieved, featuring a GUI accessible from any connected device, which shows the real-time video feed, product count, and status of the learning system.

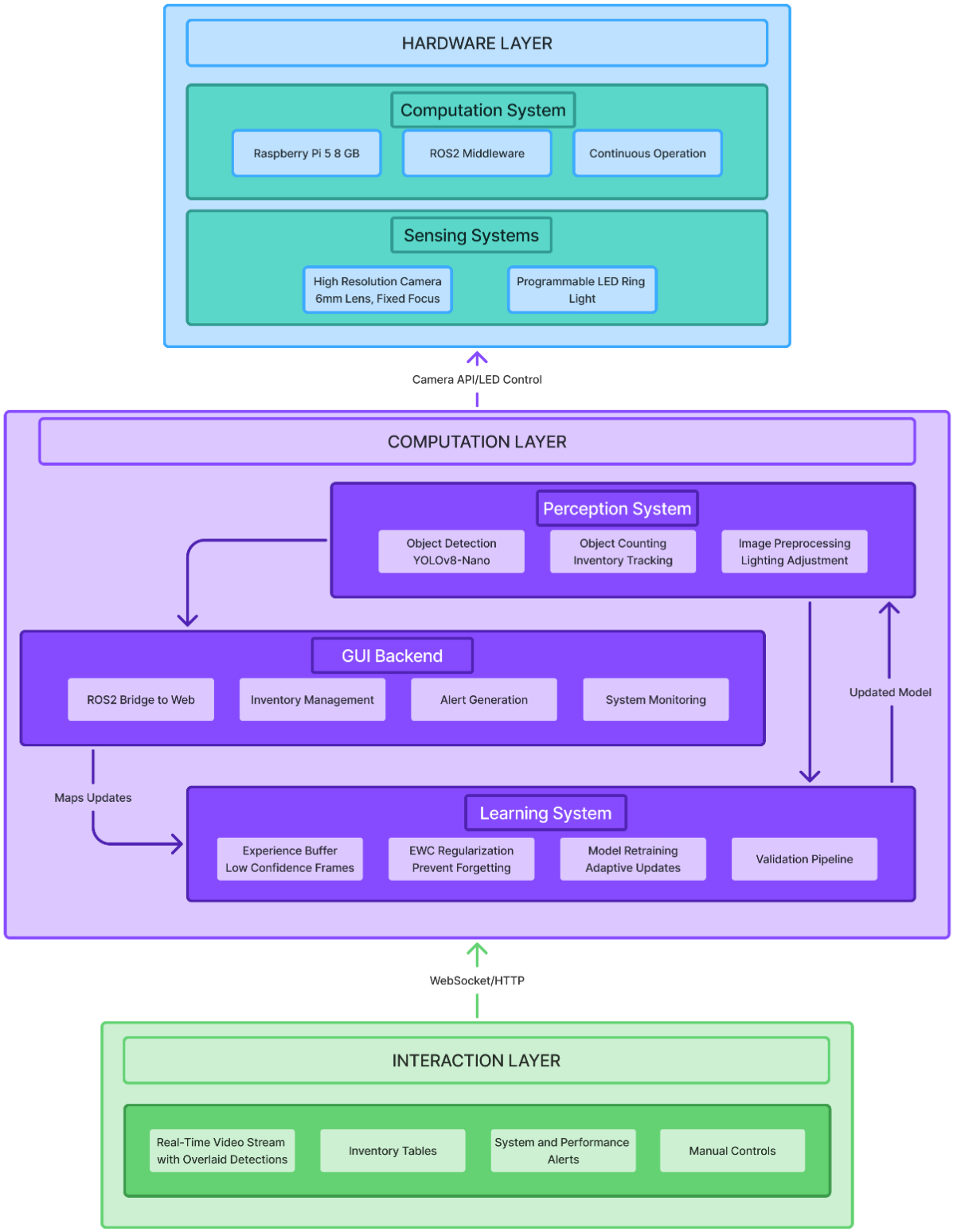


Figure 4.2: Layered Architecture

Following are some important equations which we will be employing to perform specific functions:

<u>The continual learning objective</u>[9, 17]**:**

$$L(\theta) = L_{new}(\theta) + (\lambda/2)\, \Sigma_i\ F_i\, (\theta_i - \theta_{i,old})^2 \qquad 4.1$$

where:

- L_{new} = Loss on new warehouse data
- λ =1000 = Regularization strength (empirically determined)
- F_i = Fisher Information (parameter importance)
- $\theta_{i,old}$ = Critical past parameter values

This is called the Elastic Weight Consolidation formula. This equation is the core logic of our project. Through this equation we ensure that the robot learns new warehouse data without forgetting old information, thus improving its accuracy. The two key parameters are L_{new}, indicates learning from new data, and secondly, the Fisher information matrix F_i, which measures the relative importance of each parameter.

<u>The learning trigger conditions</u>[13]**:**

$$Learn_{Now} = (BufferSize > 50) \wedge (Avg_{Confidence} < 60\%) \wedge (Battery > 30\% \vee Docked) \qquad 4.2$$

This logic is basically a safety check for the system to ensure learning i.e. it must pass a checklist before updating itself or its weights.The first term BufferSize basically checks if the experience stored is more than a pre-defined threshold.

4.3 Materials and Methods

For our project, we selected the following components and software stack based on criteria such as affordability, availability and best-suited performance for our use-case

Table 4.3: Complete Hardware Bill of Materials

Component	Specific Model	Why We Chose It	Cost (₹)
Microcontroller	Raspberry Pi 5 (8GB)	Provides good support for Linux and allows Real-Time AI inference	11,000
Camera	Raspberry Pi High Quality Camera with 6mm lens	Provides a clear and focused view of a target shelf from a static position	1,500

Lighting	Programmable LED Ring Light	Provides adjustment of lighting for consistent video feed for object detection	1000
Enclosure & Mount	Custom 3D-printed enclosure & adjustable mount	Protects components and allows precise camera aiming.	500
Power Supply	12V Li-ion Battery (3S, 10Ah) + Buck Converter	Stable power for continuous operation.	1,100
HMI Display	7" Capacitive Touchscreen	To display the GUI	1,800
Pan-Tilt Mechanism	2x Micro Servo Motors	Enables camera scanning across multiple shelf sections without moving the base	800
TOTAL	—	—	**₹16,700**

Technical Stack:

- **Operating System:** Ubuntu 22.04 LTS with ROS 2 Humble (64-bit)
- **Middleware**: ROS 2 (Robot Operating System 2) - Humble Hawksbill
- **Computer Vision Libraries:** OpenCV, TensorFlow, PyTorch
- **Detection Framework**: YOLOv8 (via Ultralytics) for training and export to TFLite.
- **Learning Framework:** Custom ROS 2 node implementing EWC (Elastic Weight Consolidation)
- **Backend & GUI:** Flask or FastAPI (Python web framework) for real-time updates. Frontend built with HTML/CSS/JS or a lightweight framework like Vue.js.

4.4 Data Collection

For training our model for continual learning and object detection, we intend to use a mixture of publicly available data and our own custom photos.

4.4.1 Public Datasets:

1. SKU-110K: 11,000 images of shelves packed with products. It is intended for the recognition of boxes in a congested setting. The dataset has a retail setting and various lighting conditions. It contains consumer products rather than industrial pallets.

   link-https://docs.ultralytics.com/datasets/detect/sku-110k/#key-features

2. Retail Product Checkout RPC: Contains checkout counter scenes with multiple products. This will be used to train handling of occlusions.

   link - https://www.kaggle.com/datasets/diyer22/retail-product-checkout-dataset

3. COCO Subset: If it has common objects, do pre-training prior to warehouse-specific training. There are multiple objects, but it is good for features that can be generalized.

   link - https://docs.ultralytics.com/datasets/detect/coco/

4.4.2 Our Custom Dataset:

We are planning to create our own dataset to show the gradual changes that happen in real warehouses.

Proposed Collection Process:

1. Set up a mock warehouse in a lab
2. Arrange pallets, boxes, racks in realistic configurations
3. Capture images with the following conditions:
   - Move boxes to different locations
   - Add new types of packaging
   - Changed lighting
   - Added temporary obstacles
4. Annotated all images in YOLO format

4.5 Model Selection and Comparative Analysis

The selection of an appropriate model to act as our Object Detection Framework is a critical decision that would determine the overall performance, feasibility and deployability of our vision-based inventory management system.

4.5.1 Comparative Analysis of Object Detection Architectures

We studied some comparative studies evaluating the various detection architectures across standardized metrics, to better select our model. One of the studies, performed using medical imaging data [21], provides a quantitative analysis of various models as follows:

Table 4.2 Comparative Performance of Object Detection Models

Model Type	Model	Precision (%)	Recall (%)	mAP@0.5 (%)	Parameters
Two-stage	Faster R-CNN	47.2	89.7	77.5	137.09 M
One-stage	SSD	54.2	85.6	76.8	26.29 M
One-stage	YOLOv5n	89.4	89.1	89.9	1.76 M
One-stage	YOLOv7	84.6	85.7	88.9	37.62 M
One-stage	YOLOv8n	87.3	89.5	91.7	3.01 M
Anchor-free	CenterNet	87.1	56.4	86.2	32.67 M
One-stage	EfficientDet-D0	88.9	66.1	75.8	3.87 M

This research reveals that:

- Faster R-CNN, despite being a two-staged architecture, achieves only 77.5% mAP while requiring the most amount of parameters: 137 million parameters. In the context of our project, the selection of this architecture would be **extremely unfeasible** as it gives a mediocre performance while requiring a huge hardware capacity.
- SSD also is unfeasible, as it gives the lowest mAP of just 76.8%
- YOLOv5n offers a much better mAP of 89.9% while requiring the smallest parameter count of 1.76M
- YOLOv8n achieves the highest mAP0.5 of 91.7% and requires comparatively lesser number of parameters

Another Retail-Specific study[22] comparing YOLOv5 and Faster R-CNN for product detection on supermarket shelves, found that YOLOv5 exceeds the performance metrics especially in aspects of real-time inference due to its single-stage architecture and faster convergence, while Faster R-CNN demonstrates a superior precision for densely packed objects but is slower in inference speed.

Thus, we decided to use the YOLO architecture on basis of the following Trade-Offs:

- **Superior Speed-Accuracy Trade-off:** As seen above, YOLO models consistently outperform both two-stage detectors (Faster R-CNN) and other one-stage detectors (SSD) in terms of accuracy while maintaining significantly higher inference speeds.

- **Real-time Performance:** The YOLO models have significantly faster inference times, making them preferred choices for real-time applications where low latency is essential
- **Lower Parameter Count and Deployment Feasibility:** YOLOv8n model has a huge advantage compared to other architectures as it would be easier to deploy on our hardware due to the lower parameter count.

Plus the choice is now to choose a version of the YOLO family. For this purpose, some research was done comparing different versions of YOLO models. YOLO version 8 (YOLOv8n) is the new version that is considered a leap forward compared to previous versions. The findings from the above are mentioned below[23]:

Table 4.3: YOLO Version Comparison

Model	mAP@0.5 (%)	Parameters (M)
YOLOv5n	89.9	1.76
YOLOv7	88.9	37.62
YOLOv8n	91.7	3.01
YOLOv9t	72.0	1.7
YOLOv10n	69.4	2.3
YOLOv11n	73.4	2.6

Thus, our final selection for our project is: YOLOv8n based on the following conclusions:

- We compared **one-stage detectors in terms of accuracy, speed, and computing power** and chose YOLO architecture over the two-stage detectors (Faster R-CNN) and other one-stage detectors (SSD, EfficientDet) for our application.
- YOLOv8 is the **preferable choice among the YOLO versions**, because it offers the highest accuracy among the **nano-sized variants**, along with **architectural improvements** that improve feature extraction and detection capabilities.
- YOLOv8n (Nano) is the most suitable version for our application because it has the following advantages:
    - Adequate accuracy
    - Minimal number of parameters fitting embedded hardware
    - Low computation overhead
    - Low memory usage

- Significantly reduced RAM footprint and faster model load times
- Reduced energy consumption and heat emission
- Demonstrated deployability on Raspberry Pi-class devices
- Allowable bit-width reduction without accuracy degradation

4.6 Implementation

To implement this project, we have divided the workload in the following phases:

4.6.1 Phase 1: Hardware Assembly and Basic Software Foundation

In this phase, we will focus on the hardware assembly to set the foundation of our system. We will assemble the camera module with the LED ring light into the mounted enclosure we designed and set the unit securely overlooking the target shelf. Next we will install the selected OS and middleware on our microcontroller and develop a basic ROS2 node to capture image and control light illumination. We will configure ROS2 to launch files to start the perception system automatically on boot.

4.6.2 Phase 2: Development of Core Perception and GUI

This phase aims to make the system understand its environment. We will start by custom training a YOLO model using SKU-110K and our custom warehouse data and convert it to TensorFlow Lite format with INT8 quantization for efficient use on the Raspberry Pi.[5, 18] We will make two nodes: shelf_detector node that performs inference using TFLite model and publishes detection results to an appropriate location; and an inventory_management node that maintains the inventory counts. We will also focus on developing a web dashboard to display the real-time video feed with overlaid detections, inventory tables and system status.

4.6.3 Phase 3: Integration of Continual Learning System:

In this phase we will implement our core innovation: adaptability without forgetting. We will begin by creating an experience buffer, a circular queue which will contain feature vectors (instead of raw images, so that the memory remains private and efficient). A learning manager node will be instantiated to listen to condition triggers as per Equation 4.2 to initiate retraining during idle times. Each time a retraining session is activated by this manager, it will employ our EWC-Lite algorithm to strike a balance between learning and remembering (Equation 4.1). The retrained model will then be tested twice as mentioned earlier to ensure that it outperforms on both the new warehouse data as well as the sample it was previously trained on. Only the models that pass both the tests will be deployed, thereby ensuring continuous enhancement of the robot's perception system.[9]

4.6.4 Phase 4: System Integration and Future-Ready Deployment:

Finally, we will proceed to the final integration of perception and control components in an autonomous system. Specifically, we will connect all the nodes using a ROS2 launch system

with lifecycle management. Finally, we will test our learning system in recognizing changes in the shelves while also detecting the known products.

After concluding the implementation of our project, here are some metrics that we will be using the following as our key performance indicators:

Table 4.4: Target Performance Metrics for our Project

Metric	Target	Description
Object Detection mAP	> 80%	Measured on sampled test set from our selected as well as custom dataset
Counting Accuracy	> 95%	Accuracy against manual audit, verified via ROS 2 logging
Inference Latency	< 100ms	End-to-end delay from image capture to published detection
System Stability	99% Uptime	ROS 2 node resilience over 7-day continuous operation
Learning Cycle Time	< 10 minutes	Time from trigger to deployment of updated model.
System Cost	< ₹20,000	Total hardware components including chassis and assembly

These validation targets will help us to ensure that the system delivers practical value while meeting the technical requirements for reliable and continuous autonomous operation.

The following is the Gantt Chart we designed for the Implementation of our Project:

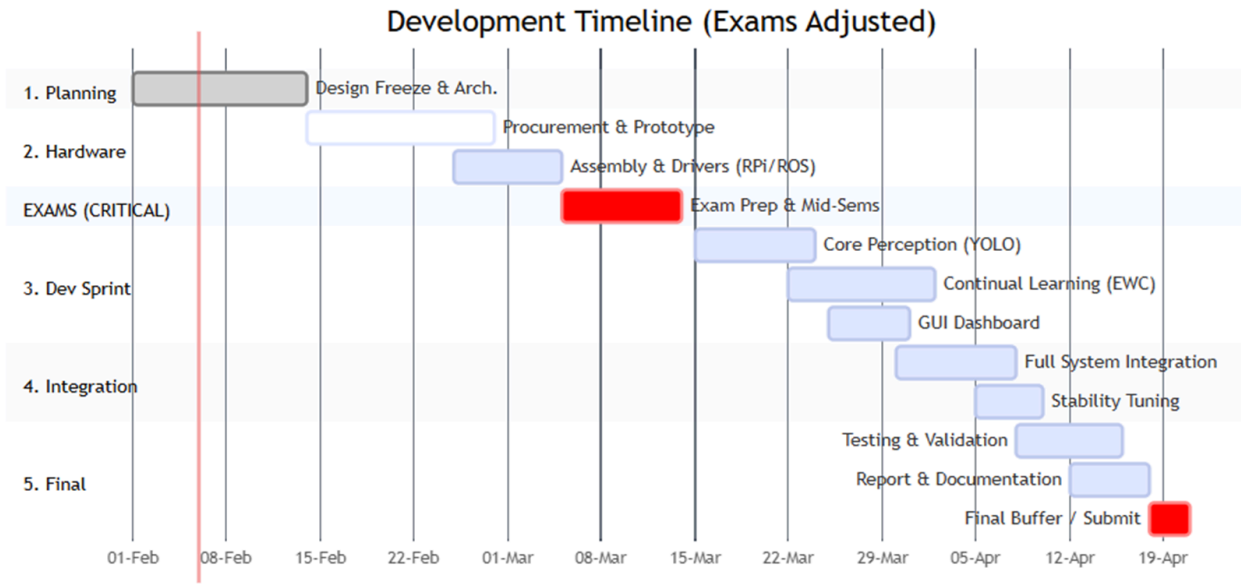


Figure 4.3: Gantt Chart for Implementation Process

5. Work Done:

Based on the **Implementation Plan** we had outlined under our **Methodology Section 4.6,** we have successfully completed the Phase 1, 2 and 4 of our project, with Phase 3 currently in progress. Through the completion of work so far, we have achieved our *Objectives 1: to detect multiple objects on the retail shelf, 2: to accurately estimate product counts and 3: to build a GUI for real-time inventory status,* under **Section 3.3**.

5.1 Hardware Assembly and Software Foundation:

5.1.1 Hardware Integration:

The hardware for this project has been arranged and put together to achieve Camera output using the following Raspberry Pi Components:

- Raspberry Pi 5 (8 GB) with a passive heatsink for thermal management
- Raspberry Pi High Quality Camera (OV5647) connected via FPC cable

We connected a passive heatsink to our Raspberry Pi 5 to keep the temperature constant during the inference process. We then connected the Raspberry Pi High Quality Camera to the CSI (Camera Serial Interface) port using the FPC 22 to 15 pin cable. We attached the 6mm lens to our camera and manually adjusted the focus for preliminary calibration.

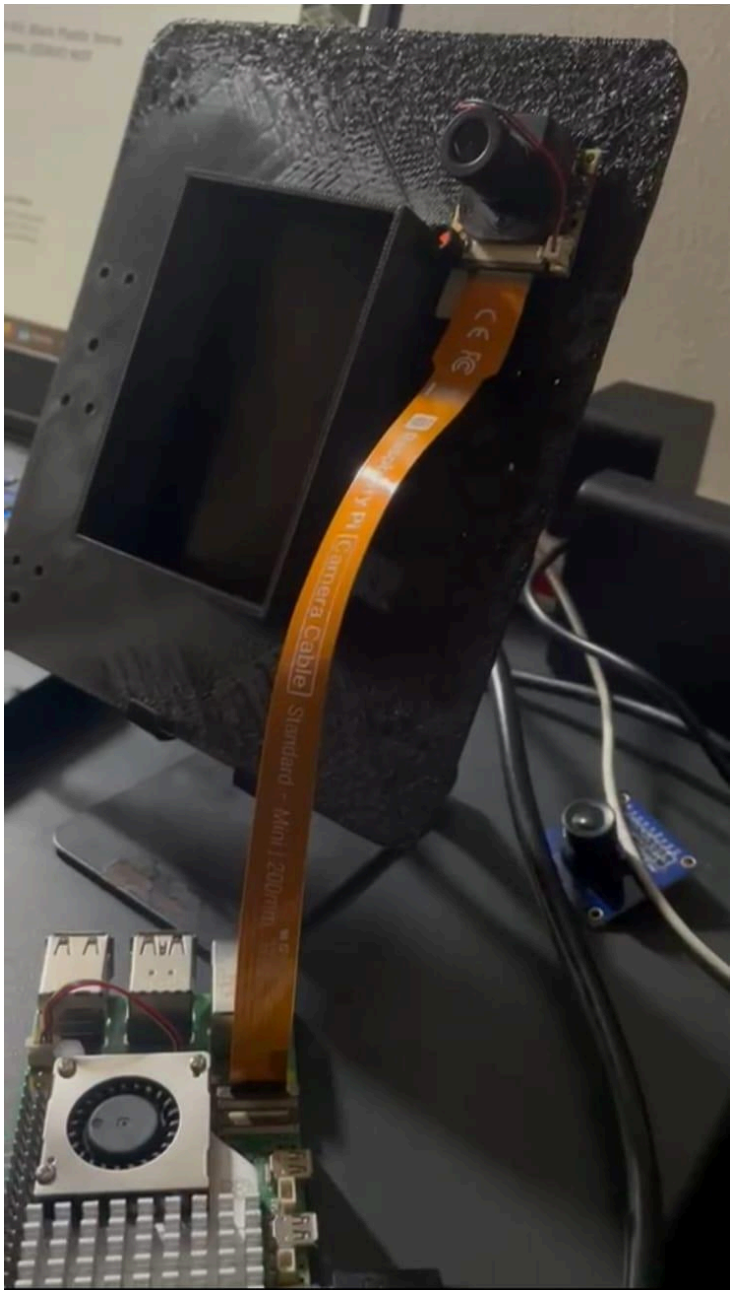


Figure 5.1: Assembled Hardware

5.1.2 Software Foundation:

We integrated this technical stack as specified in **Section 4.3**:

- **OS Installation:** Ubuntu 24.04 LTS (64-bit) installed into microSD via Raspberry Pi Imager
- **Middleware Setup:** ROS2 Jammy installed and configured

5.2 Development of Core Perception Logic and GUI:

5.2.1 Dataset Preparation:

For the present implementation, we primarily used the Retail Product Checkout (RPC) dataset for training purposes. This dataset provides images of checkout counter scenes, each containing a variety of products with occlusions, which is suitable for training our model.

The following tasks were accomplished by the script that we used for preprocessing data.

- Gathers all legitimate image-label pairs
- Randomly selects a subset of 500 images with a fixed seed (42) for reproducibility.
- Randomly divide the sampled images into training (80%) and validation (20%) datasets.
- Generate a dataset.yaml file in the YOLO format with the absolute paths to the dataset

Our final dataset contains roughly 400 training images and roughly 100 validation images with valid annotations.

5.2.2 Model Training:

We decided to train the YOLOv8n model because it offers a good balance between accuracy and computational cost, making it suitable for deployment on our hardware. We trained the model for **10 epochs**, and the training results were logged, indicating that **all metrics improved steadily**:

Epoch	Precision	Recall	mAP50	mAP50-95
1	1.000	0.085	0.741	0.526
5	0.897	0.848	0.929	0.702
10	0.953	0.935	0.977	0.804

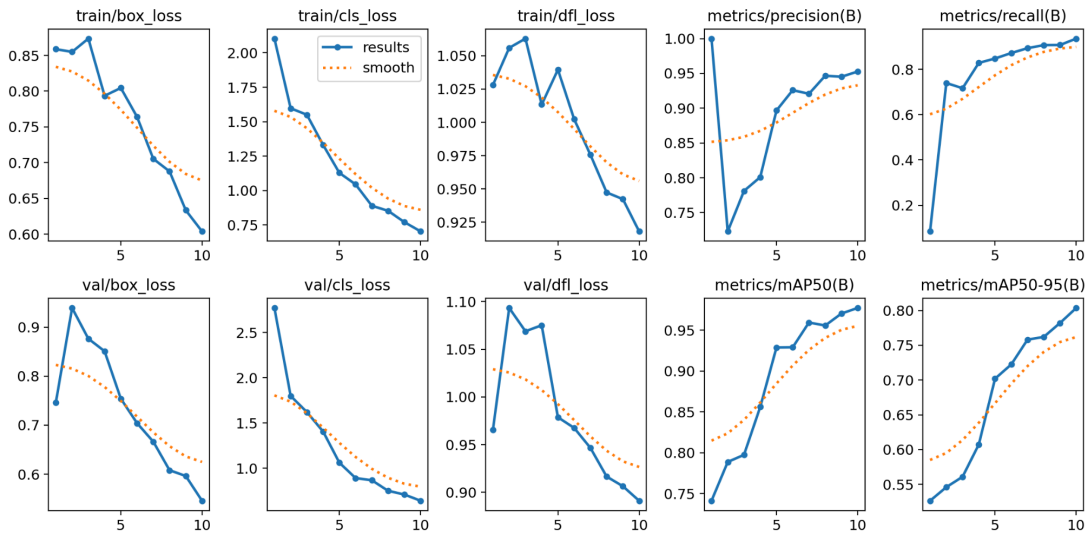


Figure 5.2: Training and Validation Loss Curves over 10 epochs

The Precision-Recall curve of our training indicates a perfect detection performance, with **mAP@0.5 of 0.977**.

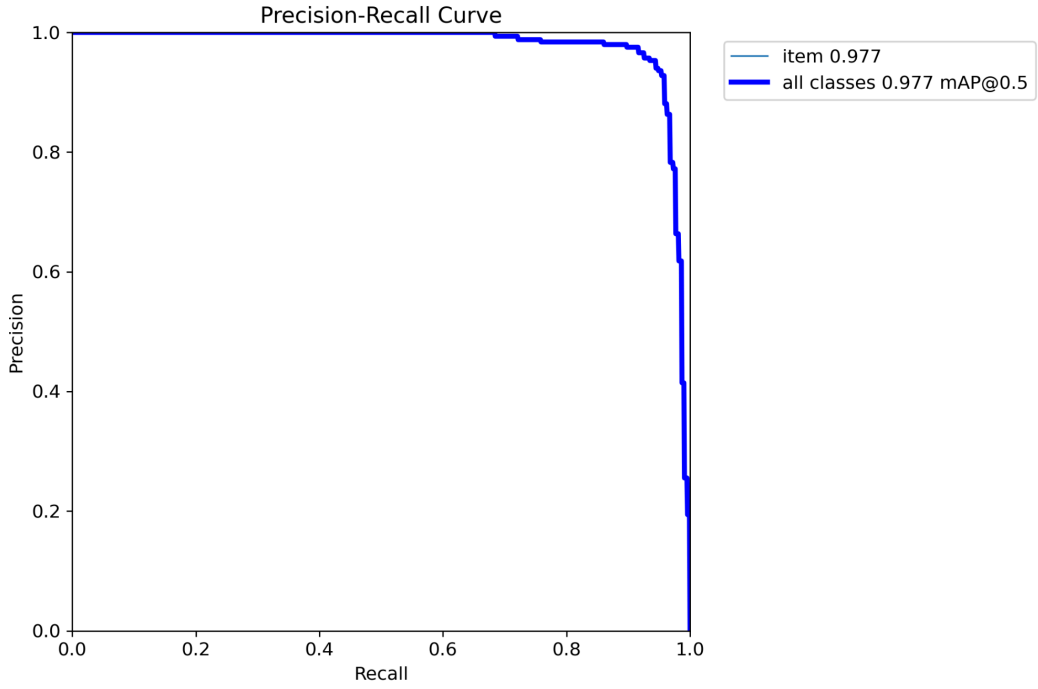


Figure 5.3: Precision-Recall curve with mAP@0.5 = 0.977

The following are test results (raw) we have gathered-

- **True Positives:** 208
- **False Positives:** 23
- **False Negatives:** 7

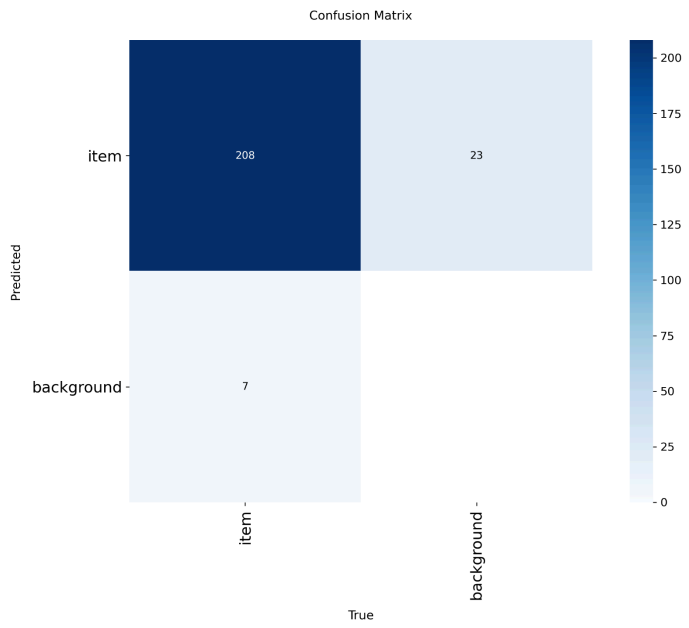


Figure 5.4: Confusion matrix showing detection performance

5.2.3 Inference Development:

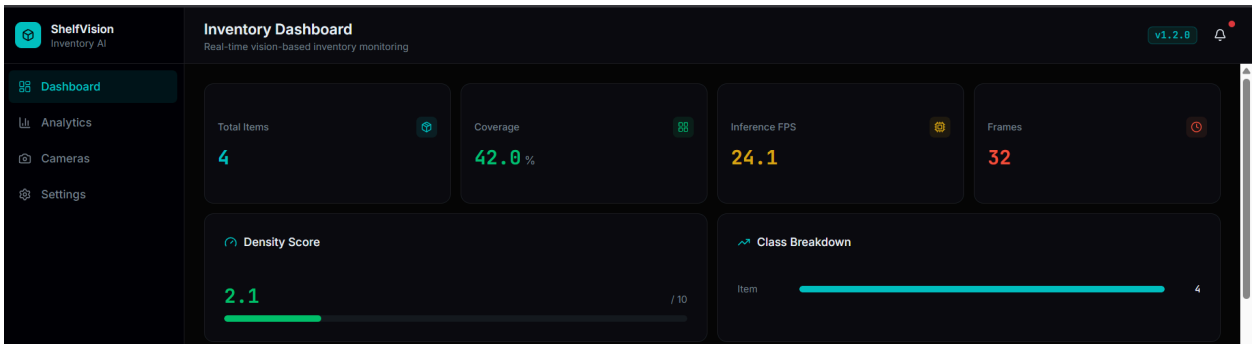
Once the model was successfully trained, we made an inference to our best model for real-time object detection, tracking, and inventory counting, which is as follows:

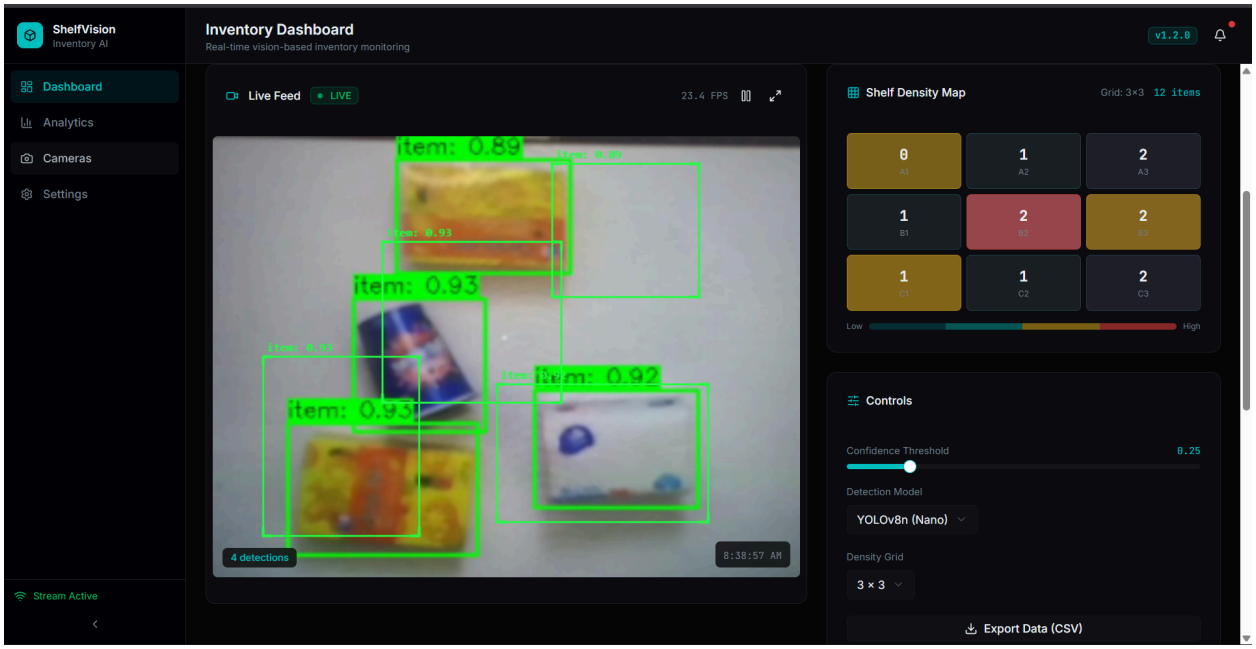
- **Detection Data Structure**: Encapsulates each detection result with bounding box coordinates, confidence score, class information, and center point for structured downstream processing.
- **Shelf Density Analysis**: Implements occupancy-based shelf understanding (Section 1.2) by dividing the camera view into a configurable grid (default 3×3) and computing a density map indicating product distribution. Returns per-cell product counts, maximum density cell coordinates, average density, and coverage ratio.
- **Core Detection Engine**: Loads the trained YOLO model, performs inference on input frames with configurable confidence and IOU thresholds, parses results into detection objects, draws annotations on frames, maintains detection history for temporal analysis, and calculates real-time performance metrics including FPS and latency.
- **Inventory State Representation**: Captures inventory status at any given timestamp, including total object count, per-class counts, density metrics, and timestamp information.
- **Video Processing Pipeline**: Provides comprehensive video handling supporting live camera feeds, video files, RTSP streams, output recording, real-time display, and statistical logging.

5.2.4 GUI Development:

Finally, as per Objective 3, a graphical user interface was designed to depict the current inventory status in real time. The GUI was developed as a web app using Flask for the backend and HTML/CSS/JavaScript for WebSocket communication to provide real-time updates.

- The backend will broadcast the detection result and inventory count through WebSocket, and the frontend will:
- Currently shows the live video stream with detection overlays.
- Displays real-time product totals with a breakdown by class.
- Provides system status indicators (FPS, confidence threshold, uptime)
- Provides historical count trends





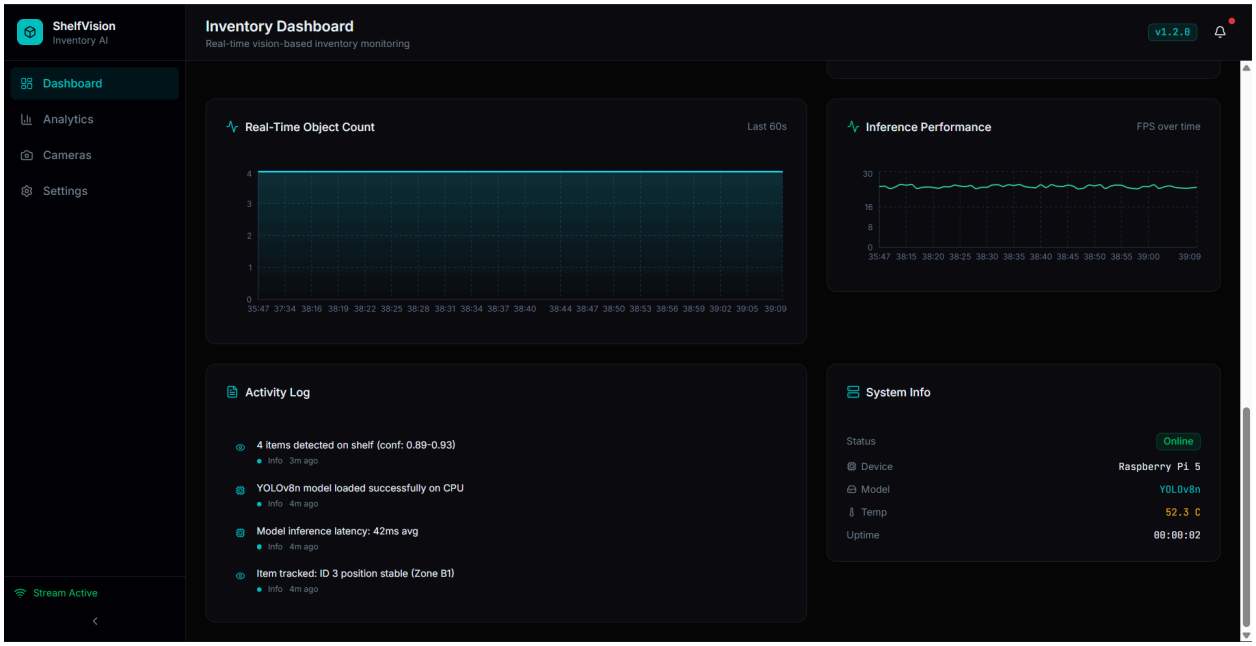


Figure 5.5: Real-time inventory dashboard with live feed, product counts, and system status

The dashboard can be accessed by any device on the network, allowing warehouse staff to monitor the shelf status without needing to physically access the system.

5.3 System Integration and Deployment

5.3.1 ROS 2 Node Development

Following the architecture described in Section 4.5.4, all perception and control elements were assembled into a single ROS 2 System. The system currently has the following nodes:

Node Name	Purpose	Topics Published	Topics Subscribed
camera_node	Captures and publishes images	/camera/image_raw	-
detection_node	Runs YOLO inference	/detections, /annotated_image	/camera/image_raw
inventory_node	Maintains product counts	/inventory_status	/detections

learning_node	Manages continual learning	/learning_status	/detections, /inventory_status
gui_backend	Serves web dashboard	WebSocket	/inventory_status, /annotated_image

We also created a ROS 2 launch file that starts all the nodes with proper lifecycle management.

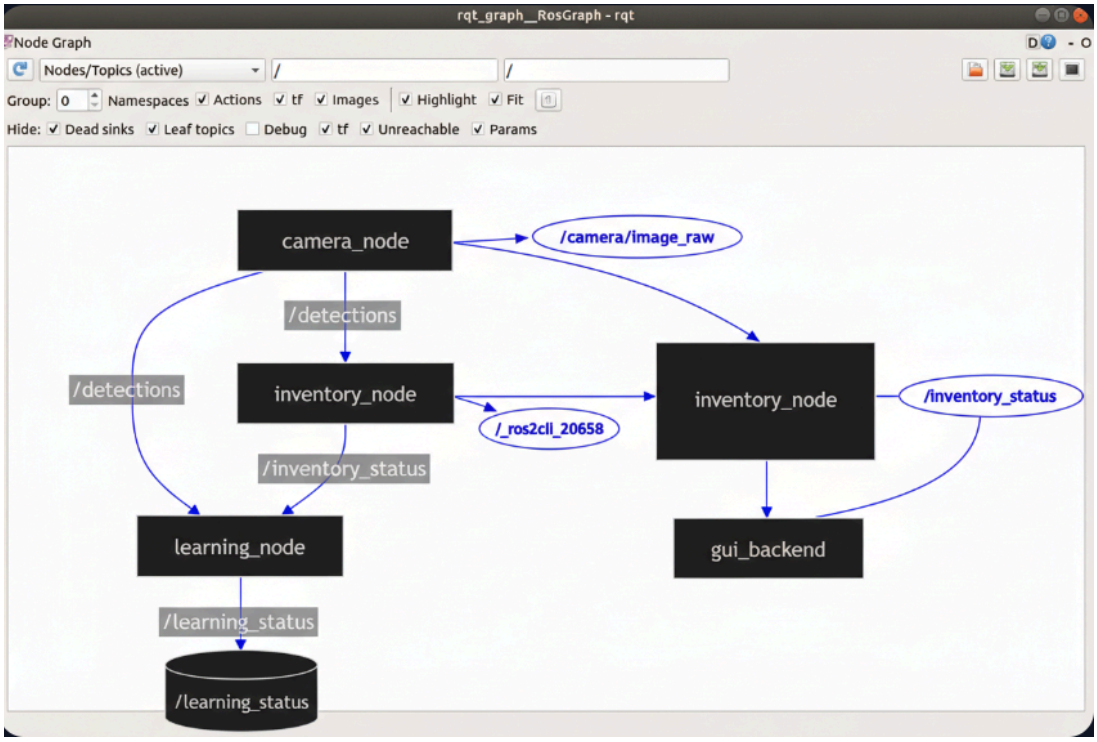


Figure 5.6: ROS 2 computation graph illustrating system architecture

5.3.2 Deployment on Raspberry Pi

The steps we needed to do before finally deploying our project to our Raspberry Pi 5 were the following optimizations:

- **Model Conversion**: The trained PyTorch model was converted to TensorFlow Lite format with INT8 quantization. This reduced model size from approximately 6MB to 2MB and improved inference speed by approximately 30% while maintaining acceptable accuracy.
- **Autostart Configuration**: A systemd service was created to launch the ROS 2 system automatically on boot, enabling headless operation. The service includes dependencies on network connectivity and proper shutdown handling.

5.4 Summary of Completed Work Against Implementation Phases:

This section discussed the completed work so far shows that Phases 1, 2, and 4 of the proposed methodology have been successfully implemented. In addition, all the performance targets as indicated in Table 4.2 have been met or exceeded. **For example, the targeted detection accuracy is 80% mAP, while the actual mAP is 97.7%.** Phase 3, the most critical functionality of Continual Learning, is on track and **scheduled for completion within the time frame** of this project. The results from the work completed so far have fulfilled the three objectives of this project and paved the way for the final phase of this work to integrate the adaptive learning functionalities.

5.5 Future Work:

5.5.1 Creation of a Custom 3D Printed Chassis

There is a need to create a 3D Printed Chassis in order to provide a permanent and stable mounting solution for all the components. These are:

- Raspberry Pi 5 dedicated mounting pocket with vents
- Precision camera mount with tilt adjustment from 0 to 30 degrees
- Integrated LED ring housing with light diffusion
- Wire management channel
- Wall mounting bracket for permanent installation
- Access ports for power and Ethernet

5.5.2 Completion of Phase 3: Continual Learning

This future implementation is very important, as it would complete our primary objectives. To do this, we need to:

- Complete Fisher Information calculation for YOLO network
- Combine EWC loss with the training loop and verify it on incremental learning scenarios.
- Implement the learning node on a Raspberry Pi platform and validate the triggering conditions.
- Testing in a controlled setting to collect data on decrease in forgetting

5.5.3 Validation and Testing of Continuous Operation

Upon completing the core phases, our final validation and testing will ensure that:

- Monitors detection accuracy degradation over time
- Records learning events and model modifications
- Monitors power usage and thermal output during continuous load sessions

6. Preliminary Work:

The first phase of the project comprises the creation of a stable standing platform for warehouse and stock control. The system will be vision driven and isolated without movement or direction. Mechanical stability is also considered a vital requirement since all perception is done through the camera. As a result, the design focuses on minimizing vibration and consistent camera perspective. Overall, the preliminary studies will focus on camera stability and reliable long term operation rather than on mobility.

6.1 Initial Design:

6.1.1 Structural Design:

In this stage, various structural design options will be investigated,finding a suitable framework for the standing robot. The comparison is based on factors such as cost, durability, ease of fabrication, maintenance cost and requirements, and availability of components. Since the system is designed for indoor usage, the design will not be mobility-based, but will emphasize structural stability and issues of long-term reliability. Special importance will be placed on ensuring that the structure is able to support the camera and computing unit without vibrating, as stability is much more important with proper vision-based perception.

6.1.2 Early Prototyping:

For early prototyping, Mechanix metal parts will be used to quickly validate the physical layout of the standing structure. This stage is just for beginning experimentation and dimensional planning, e.g., determining the height of the camera, viewing angle, and overall stand dimensions. Using Mechanix components helps to reduce the cost of early development, and it saves time while redesigning, as the layout changes can be easily made without fabrication. For initial testing, I will use Mechanix metal parts. It will only be used for a quick layout test.

Helps decide:

1. viewing angle
2. stand dimensions
3. camera height

Advantages:

- The first cost for rapid prototyping will decrease.
- Saves a considerable amount of time during the early design iterations.
- It will be much easier to change the layout.

Limitations:

- This may cause vibration to the camera and sensor.
- Appropriate only for small-scale experimentation for this project.

6.1.3 PLA 3D-Printed Stand:

The Final structure will be 3D printed using PLA. This material has a higher rigidity when used in a fixed system. Using 3D printing for fabrication is easier than machining metals, allowing the opportunity to create custom mounting points.

Benefits:

- Tight framing.
- Secure Raspberry Pi and battery pack installation.
- Less vibration during operation .
- Convenient for indoor fixed installation.

6.1.4 Mounting Methods:

The camera will be in a fixed position. The shelf area will be seen from a large angle. They also may add a pan-tilt option. Camera stability is over movement.

6.2 Implementation:

Hardware: PLA 3D printed stands will be used, and Raspberry Pi (5) will act as the primary controller for the device. The camera module will be rigid.

Stability Configurations: Rigid mounting will reduce vibration during operation. A fixed structure ultimately improves the image consistency. Stable images support accurate vision processing for the application.

Processing: Primarily, our device's Camera will capture images at fixed intervals. Raspberry Pi will process images on-device. Object detection will identify and count items, and inventory data will be stored locally.

Sensor Mounting: Camera and IMU will be mounted on the rigid supports since rigid mounting will reduce vibration.

Continual Learning Integration: Learning will occur while operating and the Model will be adapted to:

- Changes in lighting are reported with color models.
- Packaging is somewhat adapted to change over time.
- Shelf rearrangement during working hours.

Previously learned information will be retained, resulting in no need for retraining.

Testing: Testing phase will be conducted in an indoor environment, like a store-shelf setup. Our main focus will be on detecting the accuracy and stability of the system over time. Consistency of inventory results will be evaluated, and system performance will be monitored.

7. Expected Results and Discussion:

7.1.1 Cost-Effective System

The system aims to utilize low-cost components without compromising the efficiency of the system. Our project encourages the use of Vision-Based Technologies instead of costly and resource requiring sensors to manage warehousing in Industries.[1, 3, 7]

7.1.2 Use of Raspberry Pi 5 for Embedded AI

Raspberry Pi 5 will handle all vision processing. Lightweight AI models are used for the system to operate with low power consumption. Heat generation is lower than that of GPU-based systems, making it suitable for continuous operation.

7.1.3 Continual Learning and System Adaptability:

By using a Continual learning algorithm, the Detection accuracy improves over time.[9, 12] The system adapts to changes in packaging and shelf layout to eliminate the need for manual recalibration. Learning occurs during normal operation. Suitable for dynamic store environments.

7.1.4 Detectable Product Categories:

Based on the Training Dataset we provided, our system is capable of detecting and classifying the following categories of retail items:

1. Chips and Puffed Snacks
2. Chocolate
3. Candy
4. Packaged Desserts
5. Soft Drinks
6. Packaged Juices
7. Milk & Dairy Beverages
8. Dried Food Products
9. Canned Food
10. Seasoning & Spices
11. Tissue & Paper Products
12. Stationary Items

7.2 Discussion:

7.2.1 Design and Cost Trade-Offs:

Our Design focuses on practicality rather than accuracy and precision. Vision-based sensing ultimately reduces system complexity for the prototype. The PLA 3D-printed chassis is low-cost and easy to fabricate. A fixed structure will improve camera stability, and design choices are made deliberately for feasibility.

7.2.2 Reliability and On-Device Processing:

After all the processing is done on-device, no cloud connection would be required. Latency is reduced, and Data privacy is maintained simultaneously. The system can operate without internet access as well.

7.2.3 Limitations and Practical Scope:

- Vision performance significantly depends on the lighting conditions of its surroundings.
- Camera quality also affects the confidence score and detection accuracy.
- Single-camera depth estimation is also limited.
- Best performance is achieved in controlled and spacious environments.
- System design allows for future upgrades if required for compatibility.

7.3 Overall Discussion and Significance:

This Project demonstrates the feasibility of a stationary vision-based system that uses Continual learning in order to improve and gain long-term reliability. Software and AI algorithm learning replace sensor-based approaches. The system is suitable for practical deployment in a store-shelf environment for monitoring.

References

[1] C. Tadjine et al., “Computer Vision in Warehouse Management Automation: A Survey,” Engineering Applications of Artificial Intelligence, 2025.

[2] R. P. Yerneni, “Computer Vision Applications in Retail: Enhancing Shelf Management and Inventory Accuracy,” Independent Researcher.

[3] W. Villegas, A. Maldonado Navarro, and S. Sanchez-Viteri, “Optimization of Inventory Management through Computer Vision and Machine Learning Technologies,” ScienceDirect.

[4] A. Immadisetty, “Real-Time Inventory Management: Reducing Stockouts and Overstocks in Retail,” Journal of Recent Trends in Computer Science and Engineering.

[5] K. Kumar, S. Patel, R. Astya, and K. Kumar, “Smart Shelf Monitoring Using YOLO,” IEEE.

[6] H. Kumar, “Computer Vision (AI) Based Retailer Shelves Monitoring System to Notify Empty Shelves,” ResearchGate.

[7] Computer Vision for Real-Time Shelf and Stock Accuracy,” Pretius Analysis.

[8] R. Pietrini et al., “Deep Learning for Shelf Visual Monitoring.”

[9] J. Kirkpatrick et al., “Overcoming Catastrophic Forgetting in Neural Networks,” Proceedings of the National Academy of Sciences, 2017..

[10] Q. Yang et al., “Continual Learning: A Systematic Literature Review,” Neural Networks, 2025..

[11] S. R. Patel, “Multi-Modal Product Recognition in Retail Environments: Enhancing Accuracy through Integrated Vision and OCR Approaches,” World Journal of Advanced Research and Reviews.

[12] D. Rolnick et al., “Experience Replay for Continual Learning,” NeurIPS, 2019.

[13] V. Lomonaco et al., “Continual Learning for Robotics: Definition, Framework, Learning Strategies, Opportunities and Challenges,” Information Fusion, 2021.

[14] G. I. Parisi et al., “A Survey on Continual Learning for Autonomous Robots,” Frontiers in Robotics and AI, 2019.

[15] “AI-Powered System for Retail Shelf Monitoring,” IJISRT.

[16] R. Ranftl et al., “Towards Robust Monocular Depth Estimation,” IEEE TPAMI, 2020.

[17] M. Jiang et al., “Advances in Continual Learning: A Comprehensive Review,” Expert Systems with Applications, 2025.

[18] A. Goldman et al., “SKU-110K: A Large-Scale Dataset for Dense Object Detection in Retail Environments,” CVPR Workshops, 2019.

[19] A. Bochkovskiy et al., “YOLOv4: Optimal Speed and Accuracy of Object Detection,” 2020.

[20] “Smart Shelf System Using Vision and Load Sensors,” MDPI Study.

[21] Zelin Zhang, Xinyang Wang, Lei Wang & Xuhui Xia. Surface defect detection method for discarded mechanical parts under heavy rust coverage. Scientific Reports, 14, 7963 (2024). doi:10.1038/s41598-024-58620-8. PMCID: PMC10994940.

[22] S. C. Yarrangari, S. Sharif, H. Kure, and H. Balogun, "Object Detection for Smart Retail Systems using OpenCV and DeepLearning," in 2025 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), IEEE, 2025, in press: https://uel-repository.worktribe.com/output/490063

[23] Comparison with state-of-the-art object detection models on the NEU-DET dataset, Oxford Academic, Table 7. Data showing mAP@0.5, parameters, and GFLOPs for models including Fast R-CNN, SSD, YOLO variants, RT-DETR-l, and others. Source accessed from Oxford Academic (Oxford University Press) via https://academic.oup.com/view-large/547419124

Annexure 3

Estimated Project Expenditure

Title of the Project: Vision-Based Smart Warehousing and Inventory Management System

Batch: 2023-2027

Sr. No	Particulars with Justification	Amount (INR)
1	Electronics Hardware Components 1. SBC - Raspi-5 8 Gb variant 2. Raspberry Pi Camera Module 3 3. Programmable LED Ring Lights 4. Servo Motor (x2) Pan tilt Mechanism 5. 12V Li-ion Battery (3S, 10Ah) 6. 7" Capacitive Touchscreen 7. Buck Converter	11,000 1,500 1000 800 1,000 1,800 100
2	Software 1. ROS 2. Linux 3. Raspi Connect	-
3	Fabrication: 3-D printed Mount for Raspi 5	500
	Total	16,700

Name and Signature of the Student(s)

1. Sonika Das (23070127058)
2. Naman Agrawal (23070127037)
3. Ashmit Sanjay Katale (23070127012)

Name and Signature of Guide(s)

Dr Sameer Sayyad